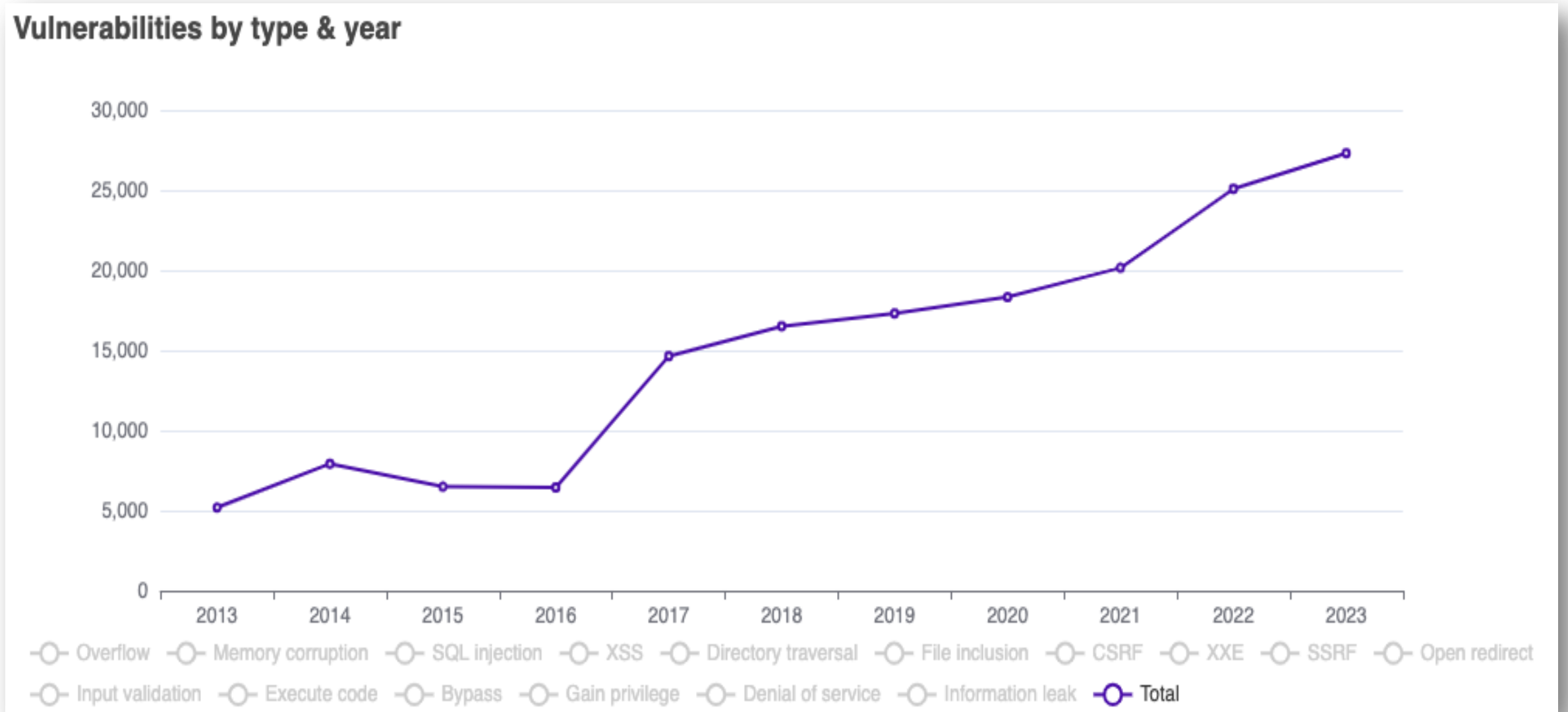


Code Clones Leads Towards Software Vulnerabilities Propagation

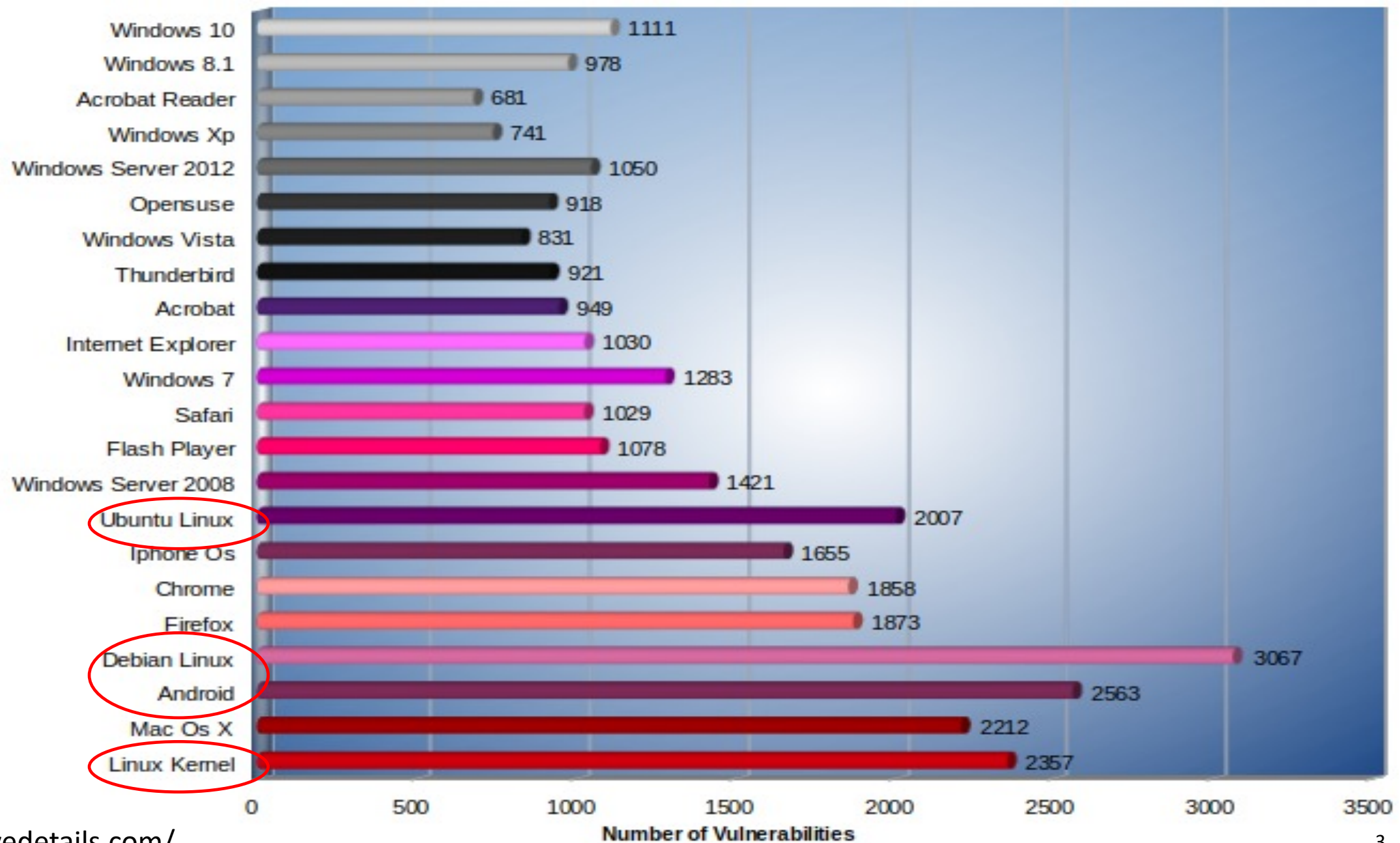
Junaid Akram, PhD



Motivation

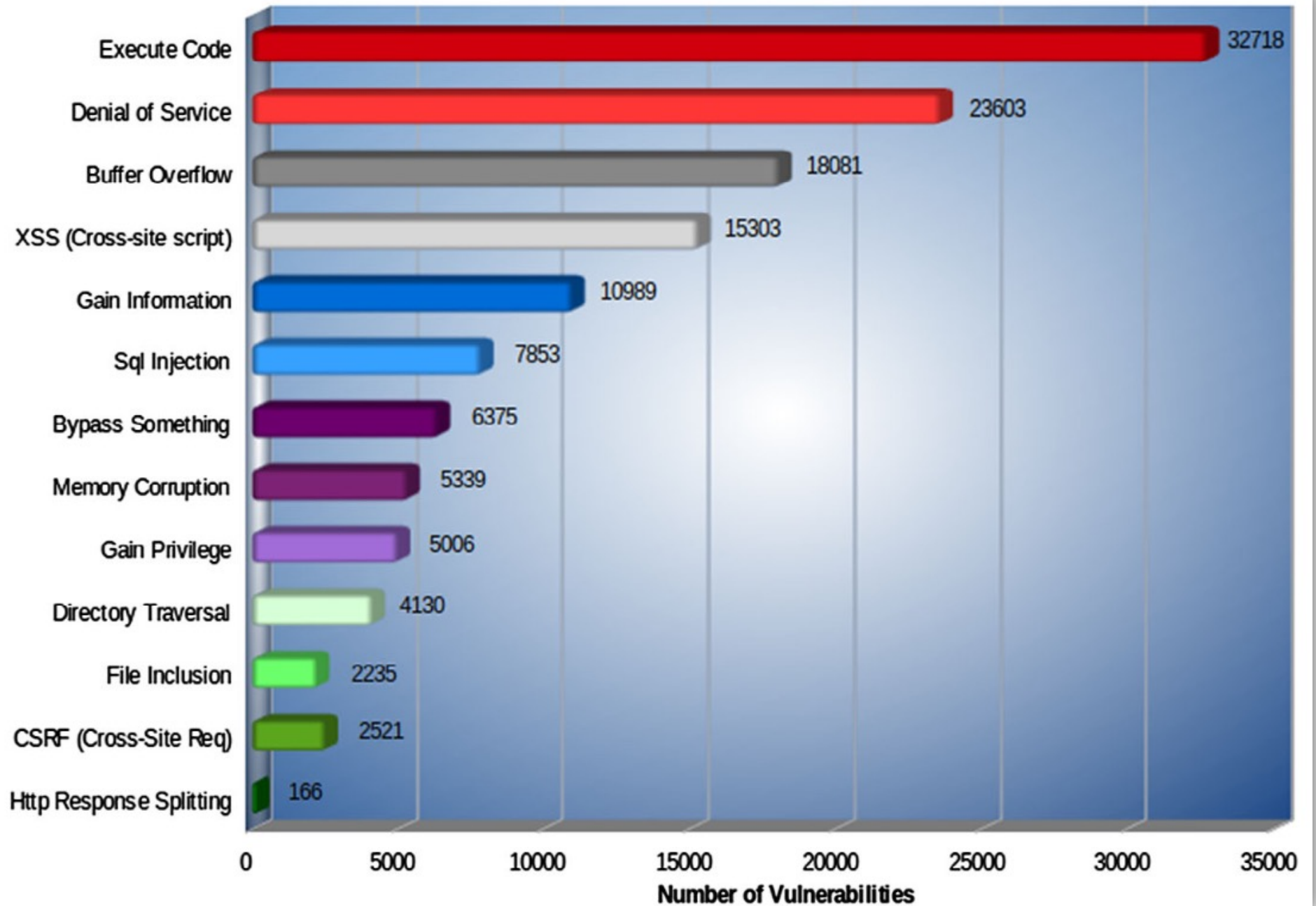


Vulnerabilities in Different Systems

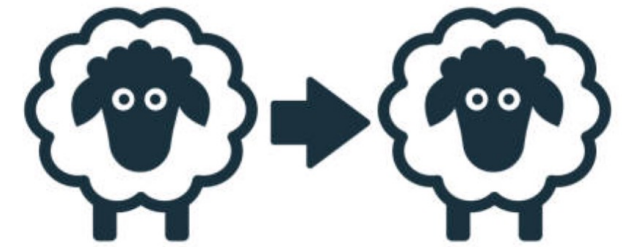


Number of vulnerabilities by Type (1999 – 2020)

Vulnerability Types



Code Clones



When a programmer **copies code** and tries to **reuse** it by pasting in his code sections with or without making minor modifications, this type of code reuse approach is called **code cloning**, and the pasted code fragment is called a clone of the original.

- ❑ There is a very significant range (**7% - 23%**) of code is cloned in large scale systems.
- ❑ On researcher has also reported that in JDK **29%** code is cloned code.

Problems....

- Code clones bring troubles in software **security** and **maintenance** and they lead to bug propagation
- Code clone detection techniques are very helpful for code **maintainability**, code **plagiarism** detection, code **verification**, **copyright** detection, security **flaws** detection, **bugs** and **malicious** software detection.
- So, it is a very important issue to detect all clones in source code accurately and as early as possible

However, it is hard to say which code fragment is original and which code fragment is copied.

Types of Clones



There are four different types of Clones...

Type-1 (Exact clones):

The identical code fragments, which are the **exact copies of code**, except blanks, layouts, white-spaces and comments.

Type-2 (Parameterized/Renamed):

Two Syntactically identical code fragments are similar except for variations in **literals**, names of **variables**, **types**, and **functions**.

Type-3 (Near miss clones/Gapped clones):

Two copied code fragments with further modifications such as **added or removed statements**, the use of **different literals**, **identifiers**, types etc. In Type-3 clone representation, it swaps two or more adjacent source code lines.

Type-4 (Semantic clones):

Two code fragments that perform the same computation but implemented by different syntactic variants. Or they are **semantically similar, without being syntactically (structure) similar**, i.e. two different code fragments perform the same task, but using different functions, statements or loops.

What is Vulnerability?

- Vulnerability is a **weakness** or a **flaw** in software system, which allows an attacker to overcome a system's security
- Vulnerable source code fragments remain **unfixed for many years** and they propagate to other systems.
- It happens, when **patch files are not propagating** to all vulnerable code clones
- Unpatched bug is a critical security problem, which should be detected and repaired as early as possible.
- **Open source software** (OSS) has increased the rate of vulnerabilities because the software developers copy the code from other systems and reuse them even without modifying it

A bug, CVE-2017-5638 was revealed in March 2017, but the hackers were using it till late May 2017 against Equifax (Lawler (2018)) and they were stealing personal information including names, birth dates and social security numbers, etc. for **143 million users in the US**

One Hunk: CVE-2018-1066 patch file

Diffstat

```
-rw-r--r-- fs/cifs/sess.c      22
-rw-r--r-- fs/cifs/smb2pdu.c  12
```

Editing performed on two files to
remove the vulnerability

2 files changed, 12 insertions, 22 deletions

```
diff --git a/fs/cifs/sess.c b/fs/cifs/sess.c
index a1118e30e7c7..dcbcc927399a 100644
```

```
--- a/fs/cifs/sess.c
+++ b/fs/cifs/sess.c

@@ -344,13 +344,12 @@ void build_ntlmssp_negotiate_blob(unsigned char *pbuffer,
/* BB is NTLMV2 session security format easier to use here? */
flags = NTLMSSP_NEGOTIATE_56 | NTLMSSP_REQUEST_TARGET |
        NTLMSSP_NEGOTIATE_128 | NTLMSSP_NEGOTIATE_UNICODE |
-       NTLMSSP_NEGOTIATE_NTLM | NTLMSSP_NEGOTIATE_EXTENDED_SEC;
-
-   if (ses->server->sign) {
+       NTLMSSP_NEGOTIATE_NTLM | NTLMSSP_NEGOTIATE_EXTENDED_SEC |
+       NTLMSSP_NEGOTIATE_SEAL;
+   if (ses->server->sign)
        flags |= NTLMSSP_NEGOTIATE_SIGN;
-       if (!ses->server->session_estab ||
-           ses->ntlmssp->sesskey_per_smbssess)
-           flags |= NTLMSSP_NEGOTIATE_KEY_XCH;
-   }
+   if (!ses->server->session_estab || ses->ntlmssp->sesskey_per_smbssess)
+       flags |= NTLMSSP_NEGOTIATE_KEY_XCH;

    sec_blob->NegotiateFlags = cpu_to_le32(flags);
```

Vulnerable file path

Patched file path

Vulnerability Detection Process using Code clones

Vulnerability Detection Main Architecture

Vulnerable Dataset



Download CVE List
(allitems.txt)

Retrieve Commit References

Patch file

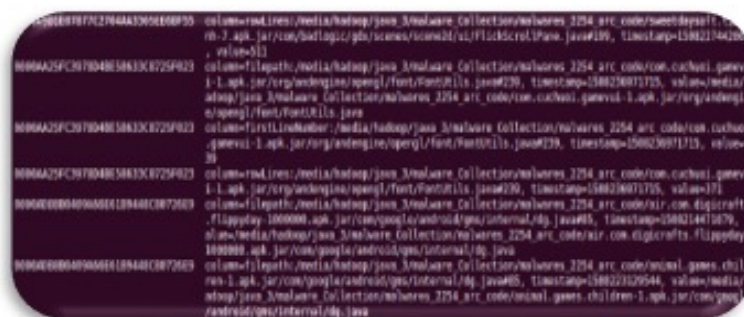
Vulnerable
source files

```
{
  id0 id1 = id2 == null || id3 == 0;
  id0 id1, id2;
  id0 id1, 0, 0; id0 id1, 0, 0;
  id0 = id1;
  id0 = id1;
  id0 = id1 id2 : id3;
  if id0 {
```

1. Preprocessing & Normalization

Loading, Transforming,
Tokenizing, Chunking

SQVDT (Architecture)



2. Feature Extraction

MD5 hash, FNV hash
Sim hash, File path,
Lines of code

Fingerprint

Indexing
DB (HBase)

Vulnerability Detection

Subject System

1. Preprocessing & Normalization
2. Feature Extraction

MapReduce ()

- Filter-1:** Compare MD5 values of test files to index into HBase
- Filter-2:** Retrieve files according to Units
- Filter-3:** Retrieve files according to Minhash
- Filter-4:** Retrieve exact clone files according to SSH file division from all nodes in Cluster

Comparison

Updating

Results

- Sorting
- Storing
- Displaying

```
66022 originpath:hdfs://192.168...:9000/sourcecode/1234567891111/linux-2.6.33-lpc313x-master/drivers/hid/hid-sunplus.c clonepath:/media/hadoop/
bf443e3e-dfe2-4c66-84fd-ef5de2b7e39f/c_code/1472246179624/bug20-2.6.31-onap-master/drivers/hid/hid-cherry.c similarity: 0.61188 1106376648
66023 originpath:hdfs://192.168...:9000/sourcecode/1234567891111/linux-2.6.33-lpc313x-master/drivers/hid/hid-sunplus.c clonepath:/media/hadoop/
bf443e3e-dfe2-4c66-84fd-ef5de2b7e39f/c_code/1474284409875/bcm963xx_4.12L.06B_consumer-master/kernel/linux/drivers/hid/hid-cherry.c similarity:
0.61188 106376648
66024 originpath:hdfs://192.168...:9000/sourcecode/1234567891111/linux-2.6.33-lpc313x-master/drivers/usb/host/xhci-hcd.c clonepath:/media/
hadoop/bf443e3e-dfe2-4c66-84fd-ef5de2b7e39f/c_code/1471022853153/android-tegra-nv-3.1.10-rel-15r7-master/drivers/usb/host/xhci.c similarity:
0.84320 6440582275
```

Vulnerability Code Extraction and Preprocessing

Patch File

```
diff --git a/drivers/media/usb/dvb-usb/digitv.c b/drivers/media/usb/dvb-usb/digitv.c
index 63134335c994..4284f6984dc1 100644
--- a/drivers/media/usb/dvb-usb/digitv.c
+++ b/drivers/media/usb/dvb-usb/digitv.c
@@ -28,22 +28,26 @@ DVB_DEFINE_MOD_OPT_ADAPTER_NR(adapter_nr);
```

```
{
- int wo = (rbuf == NULL || rlen == 0); /* write-only */
- u8 sndbuf[7],rcvbuf[7];
- memset(sndbuf,0,7); memset(rcvbuf,0,7);
+ struct digitv_state *st = d->priv;
+ int ret, wo;

- sndbuf[0] = cmd;
- sndbuf[1] = vv;
- sndbuf[2] = wo ? wlen : rlen;
+ wo = (rbuf == NULL || rlen == 0); /* write-only */
+
+ memset(st->sndbuf, 0, 7);
+ memset(st->rcvbuf, 0, 7);
+
+ st->sndbuf[0] = cmd;
+ st->sndbuf[1] = vv;
+ st->sndbuf[2] = wo ? wlen : rlen;

if (wo) {
.....
.....
}
```

Retrieved Vulnerable Code

```
{
int wo = (rbuf == NULL || rlen == 0); /* write-only */
u8 sndbuf[7],rcvbuf[7];
memset(sndbuf,0,7); memset(rcvbuf,0,7);
sndbuf[0] = cmd;
sndbuf[1] = vv;
sndbuf[2] = wo ? wlen : rlen;
if (wo) {
.....
.....
}
```

Preprocessing

```
{
id0 id1 = id2 == null || id3 == 0
id0 id1, id2;
id0 id1, 0, 0; id0 id1, 0, 0;
id0 = id1;
id0 = id1;
id0 = id1 id2 : id3;
if id0 {
.....
.....
}
```


Vulnerable files detected in 7-TB of C/C++ source code

(152,823 opensource projects, against almost 3000 vulnerable source code file)

```
15237 originpath:hdfs://192.168...:9000/CVE_code/linux_and_kernel/CVE-2018-17182/root/include/linux/vmacache.h
15238 clonepath:/media/Code2/c_code/1474977326794/linux-master/include/linux/vmacache.h similarity: 1.0
15239 originpath:hdfs://192.168...:9000/CVE_code/linux_and_kernel/CVE-2018-6412/root/drivers/video/fbdev/sbuslib.c
15240 clonepath:/media/Code2/c_code/1474989698583/linux-xtensa-xtensa-ftrace/drivers/video/fbdev/sbuslib.c similarity: 1.0
15241 originpath:hdfs://192.168...:9000/CVE_code/linux_and_kernel/CVE-2018-6412/root/drivers/video/fbdev/sbuslib.c
15242 clonepath:/media/Code2/c_code/1475110236428/net-next-nuse-for-asm-upstream-v3/drivers/video/fbdev/sbuslib.c similarity: 1.0
15243 originpath:hdfs://192.168...:9000/CVE_code/linux_and_kernel/CVE-2018-13406/root/drivers/video/fbdev/uvesafb.c
15244 clonepath:/media/Code2/c_code/1474943052043/Explay_A350_kernel_source_code-master/drivers/video/uvesafb.c similarity: 1.0
15245 originpath:hdfs://192.168...:9000/CVE_code/linux_and_kernel/CVE-2018-10689/root/btt/devmap.c
15246 clonepath:/media/Code2/c_code/1477340503128/s-blktrace-master/blktrace-userspace/btt/devmap.c similarity: 1.0
15247 originpath:hdfs://192.168...:9000/CVE_code/git_qemu/CVE-2017-6505/qemu.git/hw/usb/hcd-ohci.c
15248 clonepath:/media/Code2/c_code/1476838765540/qemu-xtensa-xtensa-new-ops/hw/usb/hcd-ohci.c similarity: 1.0
15249 originpath:hdfs://192.168...:9000/CVE_code/linux_and_kernel/CVE-2017-2647/root/net/ceph/crypto.c
15250 clonepath:/media/Code1/c_code/1474468871146/bcache-dev-bcache-dev/net/ceph/crypto.c similarity: 1.0
15251 originpath:hdfs://192.168...:9000/CVE_code/linux_and_kernel/CVE-2018-11506/root/drivers/scsi/sr_ioctl.c
15252 clonepath:/media/Code2/c_code/1477083306399/opw-staging-vimc-devel-configfs/drivers/scsi/sr_ioctl.c similarity: 1.0
15253 originpath:hdfs://192.168...:9000/CVE_code/free_Desktop/CVE-2016-7947/root/src/XrrConfig.c
15254 clonepath:/media/Code1/c_code/1472780111339/xsrc-3.4.0/external/mit/libXrandr/dist/src/XrrConfig.c similarity: 1.0
15255 originpath:hdfs://192.168...:9000/CVE_code/free_Desktop/CVE-2016-9807/root/gst/flx/flx_fmt.h
15256 clonepath:/media/Code2/c_code/1476549929668/gst-combined-master/gst-plugins-good/gst/flx/flx_fmt.h similarity: 1.0
15257 originpath:hdfs://192.168...:9000/CVE_code/free_Desktop/CVE-2016-9807/root/gst/flx/flx_fmt.h
15258 clonepath:/media/Code1/c_code/1472225988992/fread-ink-master/graphics/x-server/packages/xpm_3.5.8/src/CrDatFrI.c similarity: 1.0
15259 originpath:hdfs://192.168...:9000/CVE_code/linux_and_kernel/CVE-2016-7425/root/drivers/scsi/arcmsr/arcmsr_hba.c
15260 clonepath:/media/Code2/c_code/1475330638026/ntrdma-pu/drivers/scsi/arcmsr/arcmsr_hba.c similarity: 1.0
15261 originpath:hdfs://192.168...:9000/CVE_code/git_qemu/CVE-2016-10155/qemu.git/hw/watchdog/wdt_i6300esb.c
15262 clonepath:/media/Code2/c_code/1475841037460/qemu-stable-2.6/hw/watchdog/wdt_i6300esb.c similarity: 1.0
15263 originpath:hdfs://192.168...:9000/CVE_code/git_qemu/CVE-2016-10028/qemu.git/hw/display/virtio-gpu-3d.c
15264 clonepath:/media/Code2/c_code/1475054155364/qemu-gnuarmeclipse/hw/display/virtio-gpu-3d.c similarity: 1.0
```


Results: Discovered Vulnerabilities

- Vulnerable files detected in **7-TB** of C/C++ source code

In total, we retrieved almost **15,000 vulnerable files** of almost **504 CVE-IDs** from **150,823 open source projects** including their different versions also.

- Vulnerable files detected in Linux-5.1.4 and Linux-5.3.7

Almost **8 files** in Linux-5.1.4 and **9 files** in Linux-5.3.7

- Android_kernel_htc_msm8974
- Oppo FindX-8.1-kernel
- Xiaomi mobile-phone-Kernel 2



Results:

Most discovered vulnerabilities in the code-set

CVE-ID	CVSS_score	Vulnerability_type	CWE-ID	Occurrence
CVE-2013-7421	2.1	–	264	2522
CVE-2011-2918	4.9	DoS overflow	399	715
CVE-2014-9644	2.1	–	264	557
CVE-2010-0291	4.6	DoS +Gain privileges	264	537
CVE-2016-7097	3.6	Gain privileges	285	307
CVE-2010-3081	7.2	Overflow +Gain privileges	119	236
CVE-2014-7822	7.2	DoS	264	231
CVE-2015-2150	4.9	DoS	264	141
CVE-2011-4324	4.9	DoS	–	131
CVE-2017-9059	4.9	DoS	399	130
CVE-2010-2943	7.9	Obtain information	200	127
CVE-2016-4565	7.2	DoS	264	123

Results: List of some open source software, in which vulnerabilities are discovered w-r-t CVSS-score

Web-URL	CVE-ID	CVSS-score
https://github.com/abligh/qemu/archive/master.zip	CVE-2016-7161	10
https://github.com/felipeqc/semi/archive/master.zip	CVE-2010-2521	10
https://github.com/tartakynov/enso/archive/updatesfx.zip	CVE-2013-1591	10
https://github.com/google/capsicum-linux/archive/procdesc-4.8.zip	CVE-2016-9313	9.3
https://github.com/google/capsicum-linux/archive/misc-4.8.zip	CVE-2016-7910	9.3
https://github.com/google/ktsan/archive/tsan.zip	CVE-2015-8962	9.3
https://github.com/OSLL/qemu-xtensa/archive/windowed-2.5.zip	CVE-2016-1568	9.3
https://github.com/libos-nuse/net-next-nuse/archive/master.zip	CVE-2018-17182	7.2
https://github.com/SM-G920Pold/arter-kernel/archive/experimental.zip	CVE-2018-17182	7.2
https://github.com/USBhost/FIRE-ICE/archive/M10.5.y.zip	CVE-2017-7374	7.2
https://github.com/ljalves/linux_media/archive/latest.zip	CVE-2017-5548	7.2

Example: CVE-2015-8558:

This CVE allows local user (guest) OS administrator to cause a DOS attack to initiate an infinite loops and CPU consumption, via isochronous transfer descriptor list.

- This CVE has the **CVSS score of 4.9**, and it was found in `/hw/usb/hcd-ehci.c` in QEMU.
- We found the same CVE vulnerable file in TQSIM (Timed QEMU-based Simulator), which is an open source simulation tool that is built on QEMU.
- TQSIM is developed by CAPLab laboratory, in Seoul National University, and it is sponsored by the SAIT (Samsung Advanced Institute of Technology).
- In this simulation tool, this vulnerable file still exists and the patch file has not been applied.
- **People are still using this tool without knowing that this is vulnerable** because its information has not been published anywhere.
- The same vulnerability has been detected in **MBA (Malware Behavior Analyzer)**, which is a QEMU-based sandbox system, dedicated to malware analysis. <https://github.com/GlacierW/MBA/tree/master/hw/usb>

```
diff --git a/hw/usb/hcd-ehci.c b/hw/usb/hcd-ehci.c
index 4e2161b..d07f228 100644 (file)
--- a/hw/usb/hcd-ehci.c
+++ b/hw/usb/hcd-ehci.c
@@ -1389,7 +1389,7 @@ static int ehci_process_itd(EHCIStruct *ehci,
 {
     USBDevice *dev;
     USBEndpoint *ep;
-    uint32_t i, len, pid, dir, devaddr, endp;
+    uint32_t i, len, pid, dir, devaddr, endp, xfers = 0;
     uint32_t pg, off, ptr1, ptr2, max, mult;

     ehci->periodic_sched_active = PERIODIC_ACTIVE;
@@ -1479,9 +1479,10 @@ static int ehci_process_itd(EHCIStruct *ehci,
     ehci_raise_irq(ehci, USBSTS_INT);
 }
 itd->transact[i] &= ~ITD_XACT_ACTIVE;
+    xfers++;
 }
 }
-    return 0;
+    return xfers ? 0 : -1;
 }
```

Empirical Evaluation

- A lot of software have cloned code inside, which are **exposed as vulnerabilities**.
- We investigate almost **150,823 wide range of projects**.
- We **analyze the root cause problem** of expanding these vulnerabilities.
- The **cloned vulnerabilities** can easily be **triggered** by some known exploits.
- **Patching these vulnerabilities is very challenging when multiple vendors are involved in this clone chain.**

According to result analysis, we classify these vulnerabilities into three categories.

- 1. Intra-project clone vulnerabilities.**
- 2. Copied-kernel clone vulnerabilities.**
- 3. Library-used clone vulnerabilities.**

1. Intra-Project Clone Vulnerabilities

- These vulnerabilities were **most often appeared vulnerabilities** in our results.
- As **we detected vulnerabilities in thousands of open source projects** of almost 7-TB (C/C++) source code.
- These projects also includes different versions of one product.
- For example, we detected vulnerabilities in two different versions of Linux (Linux-5.1.4 and Linux-5.3.7).

Linux-5.1.4

CVE-2013-7421, CVE-2014-3184, CVE-2015-3212, CVE-2016-1583, CVE-2015-4178

Linux-5.3.7

CVE-2013-7421, CVE-2014-3184, CVE-2015-3212, CVE-2017-18218, CVE-2016-1583

Thus, it shows that vulnerable source code propagate from one system to another in the form of cloned code.

2. Copied-Kernel Clone Vulnerabilities

- Linux kernel is a an oldest and largest open source software. Its distribution spread to a number of open source projects, such as Android kernel, Ubuntu, Suse, Debian, IoT devices, mobile devices, etc.
- Most of the Linux kernel-based devices are a **bit behind the patching process**.
- For example, we detected many kernel **vulnerabilities in some smartphones software** especially in OS which are kernel based.

- **Android_kernel_htc_msm8974**

CVE-2013-0310, CVE-2013-7339, CVE-2014-8884, CVE-2013-4512, CVE-2012-3430, CVE-2012-2669, CVE-2016-10318, CVE-2012-4565, CVE-2013-7027, CVE-2013-1827, CVE-2014-5206, CVE-2014-5207, CVE-2013-0216, CVE-2009-3620, CVE-2012-2373, CVE-2012-6546, CVE-2015-2150, CVE-2013-1957, CVE-2012-5374, CVE-2012-5375, CVE-2013-7263, CVE-2013-7264, CVE-2013-7265, CVE-2013-7281, CVE-2010-3081, CVE-2014-7822, CVE-2014-9644, CVE-2013-7421

- **FindX-8.1-kernel**

CVE-2018-6412, CVE-2017-16912, CVE-2014-3184, CVE-2013-7421, CVE-2017-16914, CVE-2015-3212, CVE-2017-18075, CVE-2016-1583, CVE-2015-4178, CVE-2017-15951, CVE-2018-18386, CVE-2017-17805, CVE-2017-18232, CVE-2016-9777, CVE-2017-9059, CVE-2017-11472, CVE-2018-17182

3. Library-Used Clone Vulnerabilities

- Different software directly or indirectly use some open source libraries to speedup the development process.
- In practice, it takes place frequently, as these libraries (third-party) are developed to be reused.
- For example, only VLC-player used almost 9 third-party libraries, including LAME, FFmpeg, FLAC, etc.
- We detected many projects which are using some outdated libraries or the library files which are vulnerable or not patched yet.
- Few discovered Library Vulnerabilities CVEs are displayed below as a reference.

CVE-2016-8650, CVE-2016-7914, CVE-2017-12193, CVE-2015-8948, CVE-2010-2498

Vulnerabilities Verification

There were two ways to verify the detected vulnerabilities, exploit generation, or the static analysis.

- **Patch-Based Exploit Generation**

We can create a direct exploit by exploring that particular vulnerability, i.e, denial of service, execute code, overflow, sql injection, etc.

- **Patch-Based Static Analysis** (our adopted method)

In this method, we explored the number of files on which patch files were applied against each detected vulnerability (CVE-number).



Vulnerabilities discovered in Oppo FindX-8.1-kernel

1 file changed, 1 file detected as not patched.

- originpath:/FindX-8.1-kernel-master/drivers/video/fbdev/sbuslib.c
clonepath:/linux-kernel/CVE-2018-6412/root/drivers/video/fbdev/sbuslib.c
- originpath:/FindX-8.1-kernel-master/drivers/usb/usbip/stub_rx.c
clonepath:/linux-kernel/CVE-2017-16912/root/drivers/usb/usbip/stub_rx.c
- originpath:/FindX-8.1-kernel-master/drivers/usb/usbip/stub-tx.c
clonepath:/linux-kernel/CVE-2017-16914/root/drivers/usb/usbip/stub_tx.c
- originpath:/FindX-8.1-kernel-master/drivers/acpi/acpica/nsutils.c
clonepath:/linux-kernel/CVE-2017-11472/root/drivers/acpi/acpica/nsutils.c
- originpath:/FindX-8.1-kernel-master/include/linux/vmacache.h
clonepath:/linux-kernel/CVE-2018-17182/root/include/linux/vmacache.h
- originpath:/FindX-8.1-kernel-master/include/net/netns/sctp.h
clonepath:/linux-kernel/CVE-2015-3212/root/include/net/netns/sctp.h
- originpath:/FindX-8.1-kernel-master/crypto/pcrypt.c
clonepath:/linux-kernel/CVE-2017-18075/root/crypto/pcrypt.c
- originpath:/FindX-8.1-kernel-master/fs/ecryptfs/kthread.c
clonepath:/linux-kernel/CVE-2016-1583/root/fs/ecryptfs/kthread.c

Vulnerabilities discovered in HTC-kernel 14.1

1 file changed, 1 file detected as not patched.

- originpath:/android-kernel-htc-14.1/net/ipv4/cipso_ipv4.c
clonepath:/CVE-2013-0310/root/net/ipv4/cipso_ipv4.c
- originpath:/android-kernel-htc-14.1/net/rds/ib.c
clonepath:/CVE-2013-7339/root/net/rds/ib.c
- originpath:/android-kernel-htc-14.1/drivers/media/dvb/ttusb-dec/ttusbdecfe.c
clonepath:/CVE-2014-8884/root/drivers/media/usb/ttusb-dec/ttusbdecfe.c
- originpath:/android-kernel-htc-14.1/arch/um/kernel/exitcode.c
clonepath:/CVE-2013-4512/root/arch/um/kernel/exitcode.c
- originpath:/android-kernel-htc-14.1/net/rds/recv.c
clonepath:/CVE-2012-3430/root/net/rds/recv.c
- originpath:/android-kernel-htc-14.1/tools/hv/hv_kvp_daemon.c
clonepath:/CVE-2012-2669/root/tools/hv/hv_kvp_daemon.c
- originpath:/android-kernel-htc-14.1/fs/crypto/policy.c
clonepath:/CVE-2016-10318/root/fs/crypto/policy.c
- originpath:/android-kernel-htc-14.1/net/ipv4/tcp_illinois.c
clonepath:/CVE-2012-4565/root/net/ipv4/tcp_illinois.c

Vulnerability discovered in 7-TB of Open Source Projects

```
Diffstat
-rw-r--r-- drivers/usb/gadget/rndis.c 11
1 files changed, 7 insertions, 4 deletions

diff --git a/drivers/usb/gadget/rndis.c b/drivers/usb/gadget/rndis.c
index 9689efe..6d6eaad 100644
--- a/drivers/usb/gadget/rndis.c
+++ b/drivers/usb/gadget/rndis.c
@@ -853,11 +853,14 @@ static int rndis_query_response (int configNr, rndis_query_msg_type *buf)
    // DEBUG("%s: OID = %08X\n", __FUNCTION__, cpu_to_le32(buf->OID));
    if (!rndis_per_dev_params [configNr].dev) return -ENOTSUPP;

-    /*
-     * we need more memory:
-     * oid_supported_list is the largest answer
+    /*
+     * we need more memory:
+     * gen_ndis_query_resp expects enough space for
+     * rndis_query_cmplt_type followed by data.
+     * oid_supported_list is the largest data reply
+     */
    r = rndis_add_response (configNr, sizeof (oid_supported_list));
+    r = rndis_add_response (configNr,
+        sizeof (oid_supported_list) + sizeof (rndis_query_cmplt_type));
    if (!r)
        return -ENOMEM;
    resp = (rndis_query_cmplt_type *) r->buf;
```

This vulnerability was detected in
GPL source dump for the **Cisco-WAP4410N**.

CVE-2006-1368 patch file

CVE-2006-1368 CVSS scores and vulnerability types

CVSS Score	10.0
Confidentiality Impact	Complete (There is total information disclosure, resulting in all system files being revealed.)
Integrity Impact	Complete (There is a total compromise of system integrity. There is a complete loss of system protection, resulting in the entire system being compromised.)
Availability Impact	Complete (There is a total shutdown of the affected resource. The attacker can render the resource completely unavailable.)
Access Complexity	Low (Specialized access conditions or extenuating circumstances do not exist. Very little knowledge or skill is required to exploit.)
Authentication	Not required (Authentication is not required to exploit the vulnerability.)
Gained Access	None
Vulnerability Type(s)	Denial Of Service Overflow Memory corruption
CWE ID	119

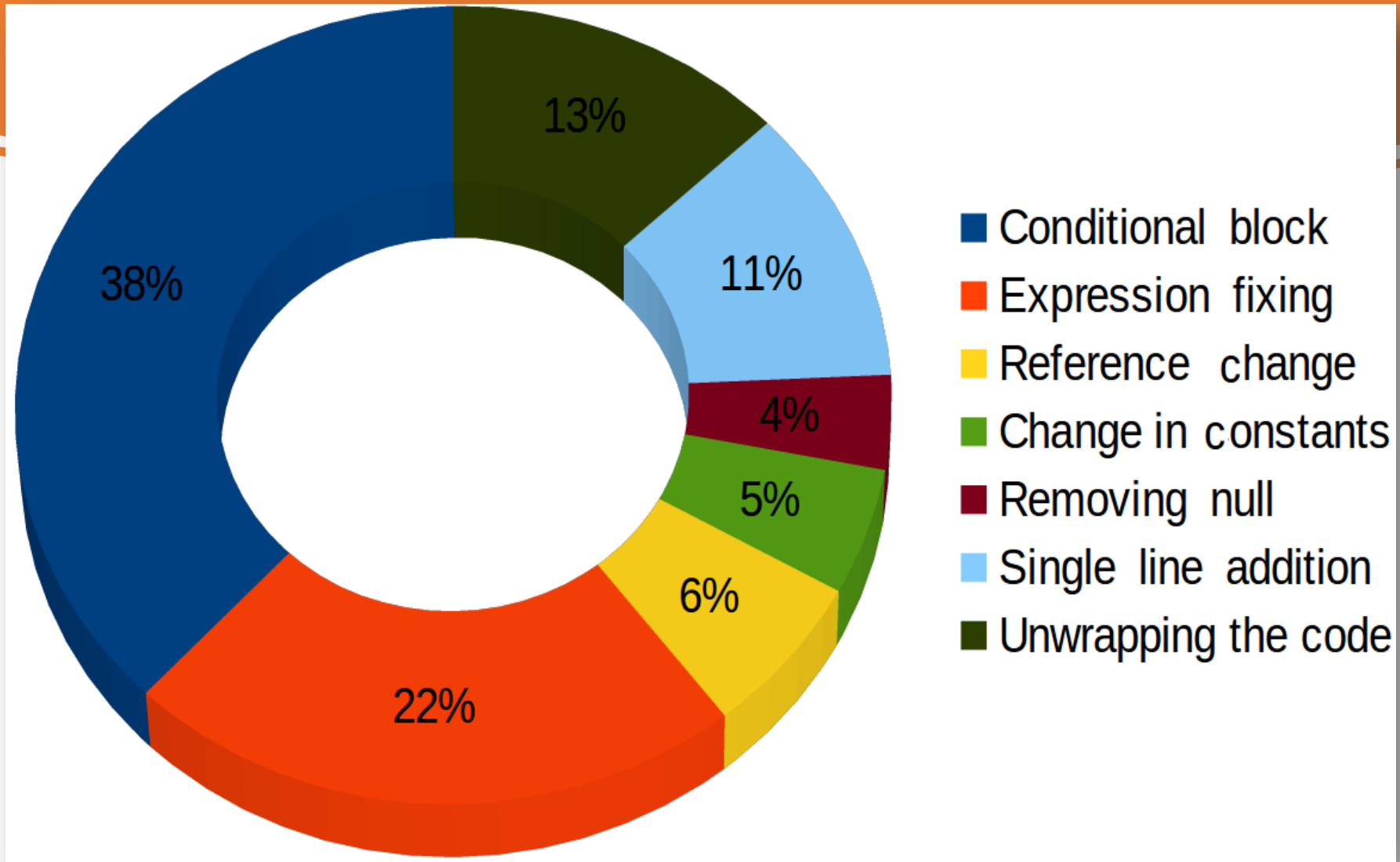
Unpatched Vulnerable Code on GitHub in 7-TB of Source Code

Even though the patch files of these vulnerabilities are already published but these projects are still not patched and the people are using them without knowing this.

- **CVE-2013-0310:** This denial of services (pointer dereference and system crash) vulnerability was first discovered in Linux kernel before V.3.4.8, in the function of `cipso_v4_validate` in `net/ipv4/cipso_ipv4.c` file. We detected almost **42** open source GitHub projects that contain this vulnerability.
- **CVE-2018-6412:** This vulnerability was found in the Linux kernel 4.15 that exist in a function `sbusfb_ioctl_helper()` of file `drivers/video/fbdev/sbuslib.c`. This is an information leakage vulnerability that allows an attacker to steal information. **23** GitHub projects still contain this vulnerability.
- **CVE-2017-11472:** This is another "obtain information" vulnerability that allow local user to obtain sensitive information from memory. This vulnerability is in a function `acpi_ns_terminate()` of a file `drivers/acpi/acpica/nsutils.c`. Almost **16** projects discovered that contain this vulnerability.
- **CVE-2016-10318:** This is a denial of services vulnerability discovered in 2016, that allow users to assign an encryption policy to a directory owned by a different user. **38** GitHub projects contain this vulnerability.
- **CVE-2017-18075:** This denial of services vulnerability exist in a file `crypto/pcrypt.c` of Linux kernel before version 4.14.13. Almost **33** GitHub projects discovered that contain this vulnerability.

For detail: <https://github.com/znd15/Vul in 7TB/blob/master/Vul in 7TB>

Repair patterns ratio in 200 patches



Thanks for your attention!

Any Question?



FAQ..

1. How software vulnerabilities propagate from one system to another system?
2. Why do the changes require in the source code through patch files?
3. Do all the repair patterns in patches remove the vulnerability issues?
4. Do the vulnerable code is identifiable, when a patch is released (Hacker's Perspective) ?
5. Am I (end users, developers, software engineers) vulnerable, by using components of known vulnerabilities?
6. Is it possible to detect unpatched (vulnerable) code fragments in my software?

